

ENGINEERING SUCCESS

VIBE CODING IM UNTERNEHMENS- KONTEXT

Wenn KI den Code schreibt — und der Mensch die Richtung vorgibt.

VORTRAGENDE

Dipl.-Inf. Minh Cuong Tran

ANDRITZ SCHULER

Jakob Ayo, B.Sc.

HOCHSCHULE ESSLINGEN

Viet Pham, B.Sc.

UNIVERSITÄT TÜBINGEN

AGENDA

9 STATIONEN, 20 MINUTEN.

Vom Konzept über drei tiefe Use Cases bis zum Ausblick.

01 Was ist Vibe Coding?

02 Wie funktioniert das?

03 10 Praxisbeispiele im Überblick

04 Highlight: Legacy-Modernisierung

05 Risserkennung mit KI
VIET PHAM

06 Fully Agentic Development
JAKOB AYO

07 Vibe Coding Patterns
MINH CUONG TRAN

08 Persönliche Projekte

09 Fazit

DEFINITION

WAS IST VIBE CODING?

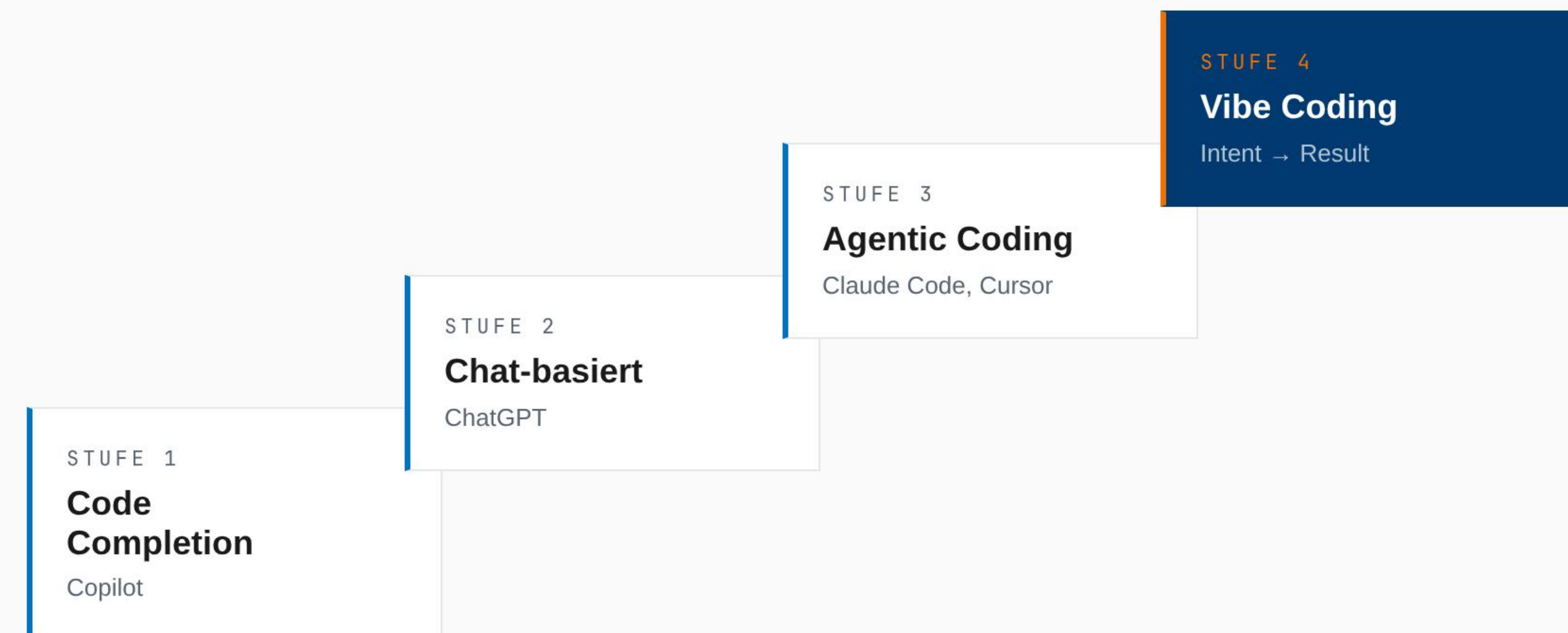
Softwareentwicklung durch natürliche Sprache — Mensch gibt die Richtung vor, KI-Agenten setzen um.

“

*There's a new kind of coding I call **vibe coding**, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists.*

— ANDREJ KARPATHY · FEB 2025

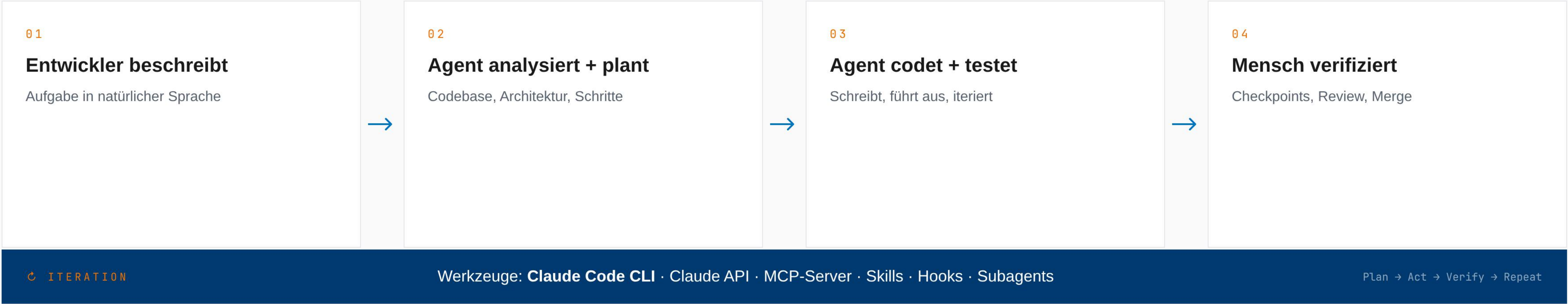
EVOLUTION DER KI-ENTWICKLUNG



WORKFLOW

HUMAN-IN-THE-LOOP

Vier wiederkehrende Schritte. KI als „Junior-Entwickler mit Superkräften“ — schnell, aber sie braucht Führung.





BANDBREITE

10 PRAXISBEISPIELE IM ÜBERBLICK

Legacy-Migration · Security · Greenfield — 3× bis 20× schneller. Alle Projekte abstrahiert.

#	KATEGORIE	TECHNOLOGIEN	BESCHLEUNIGUNG
01	WPF → React Migration	C#, XAML → React/TS	12×
02	Greybox Penetration Test	Security Assessment	3.5×
03	KI-gestützte Code Review	.NET, Python, WPF	2.5×
04	Fortran AIX → Linux Migration	Fortran, Go, Docker	18×
05	PHP 5 → PHP 8 + Go Rewrite	PHP, Go	3.5×
06	VB 64-bit Portierung	VBA, Access	20×
07	REXX → Go Modernisierung	REXX, Go	signifikant
08	SAP ABAP Modernisierung	ABAP, BRF+	PoC
09	Live Vibe Coding Workshop	JavaScript, HTML5	5×
10	IT-Inventur Automatisierung	Data Pipeline	automatisiert



LEGACY-MODERNISIERUNG

TRADITIONELL VS. VIBE CODING.

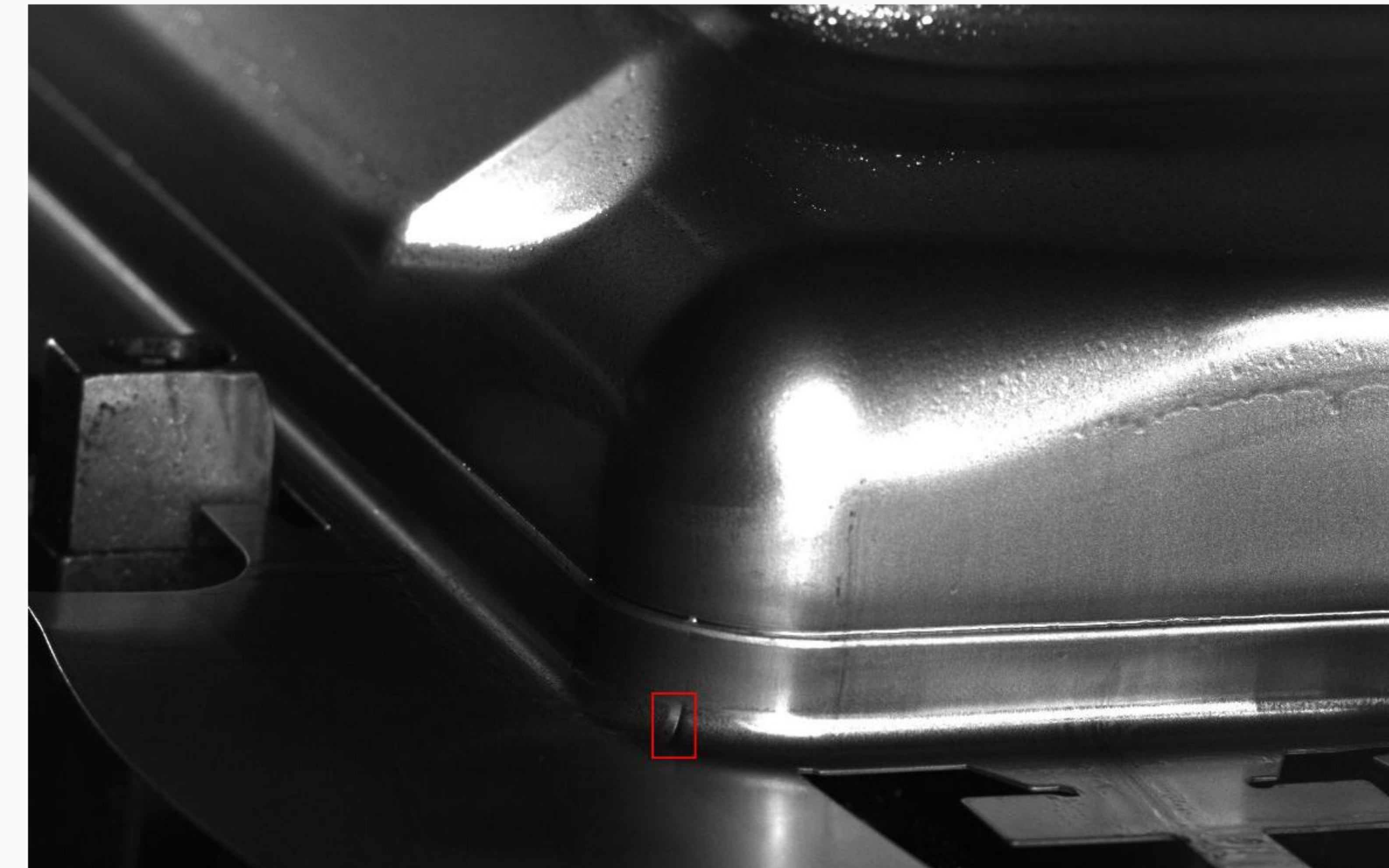
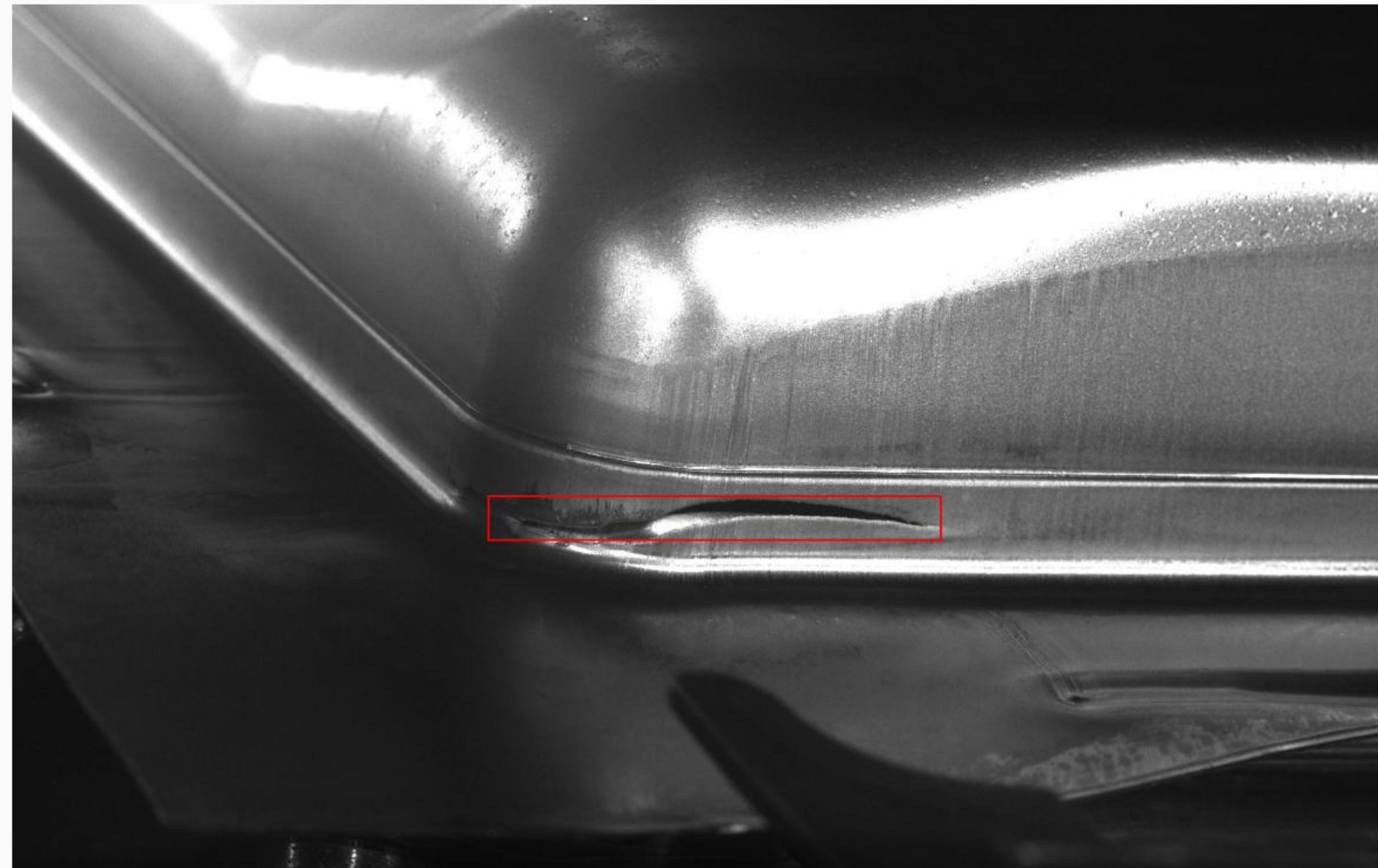
KI ist besonders stark bei „verstandener“ Transformation — klare Quelle, klares Ziel.

PROJEKT-KATEGORIE	DAUER (TAGE)	FAKTOR
Fortran (500k LOC) AIX → Linux · 15.000 Tests · Go Web-UI · Docker	Traditionell · ~90 Tage Mit KI · 10 Tage	18×
WPF → React 77 Dateien · 15 Telerik-Controls → MUI	Traditionell · ~90 Tage Mit KI · 15 Tage	12×
REXX → Go Gehaltsübertragung · 18.207 LOC Go · Prometheus · 46+ Error Codes	Traditionell · ~60 Tage Mit KI · 8 Tage	7×

USE CASE · QUALITÄTSKONTROLLE

RISSE ERKENNEN, WO DAS AUGES VERSAGT.

Pressen-Qualitätskontrolle: kleine Risse, große Flächen, gescheiterte Vorgängerprojekte.



Kleine Risse, große Flächen

Hochauflösende Aufnahmen, sub-mm Defekte über m²-Werkstücken.

Hohe False-Negative-Kosten

Ein übersehener Riss = Ausschuss bei der Folge-Charge.

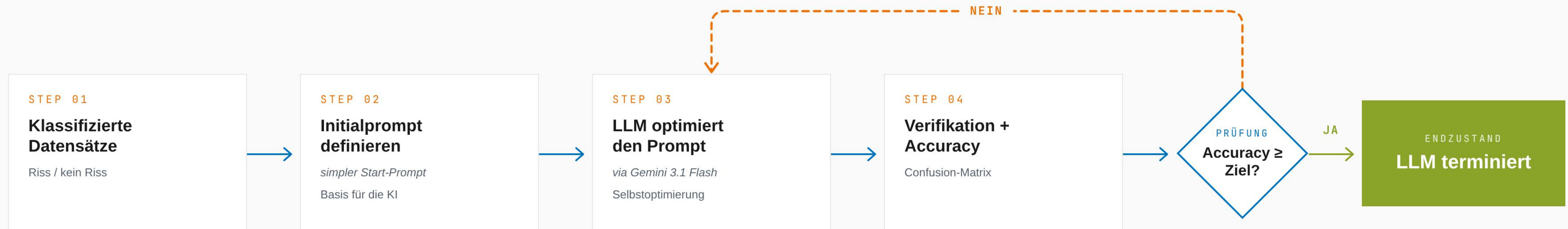
2 Jahre Vorlauf — gescheitert

Klassische CV-Ansätze und vortrainierte CNNs scheiterten an Domain-Shift.

ANSATZ · METHODIK

LOOP ENGINEERING.

Statt CNN-Training: Gemini 3.1 Flash klassifiziert, die LLM verfeinert ihre Anweisung selbst.



ERGEBNIS · ERKENNTNISSE

VOM 2-JAHRES-STILLSTAND ZUR PRODUKTIONSREIFEN LÖSUNG.

96.4%

Accuracy

98.1%

Recall

94.7%

Precision

LESSON LEARNED

Wo sich die Klassifikation im Prompt formulieren lässt, ist ein Vision-LLM eine ernstzunehmende Alternative zur klassischen CV-Pipeline.

ÜBERTRAGBARKEIT

Übertragbar auf jede visuelle Anomalie-Erkennung mit klassifizierten Beispielen — Schweißnähte, Oberflächen, Verpackung.

KONZEPT

ZWEI SICHTEN AUF VOLLAUTONOME ENTWICKLUNG.

Autonomie-Spektrum trifft Closed-Loop — der Agent korrigiert sich selbst, statt zu warten.

AUTONOMIE-SPEKTRUM



← zunehmender Verzicht auf menschliches Eingreifen

VORAUSSETZUNG 1

Task-Verifizierbarkeit

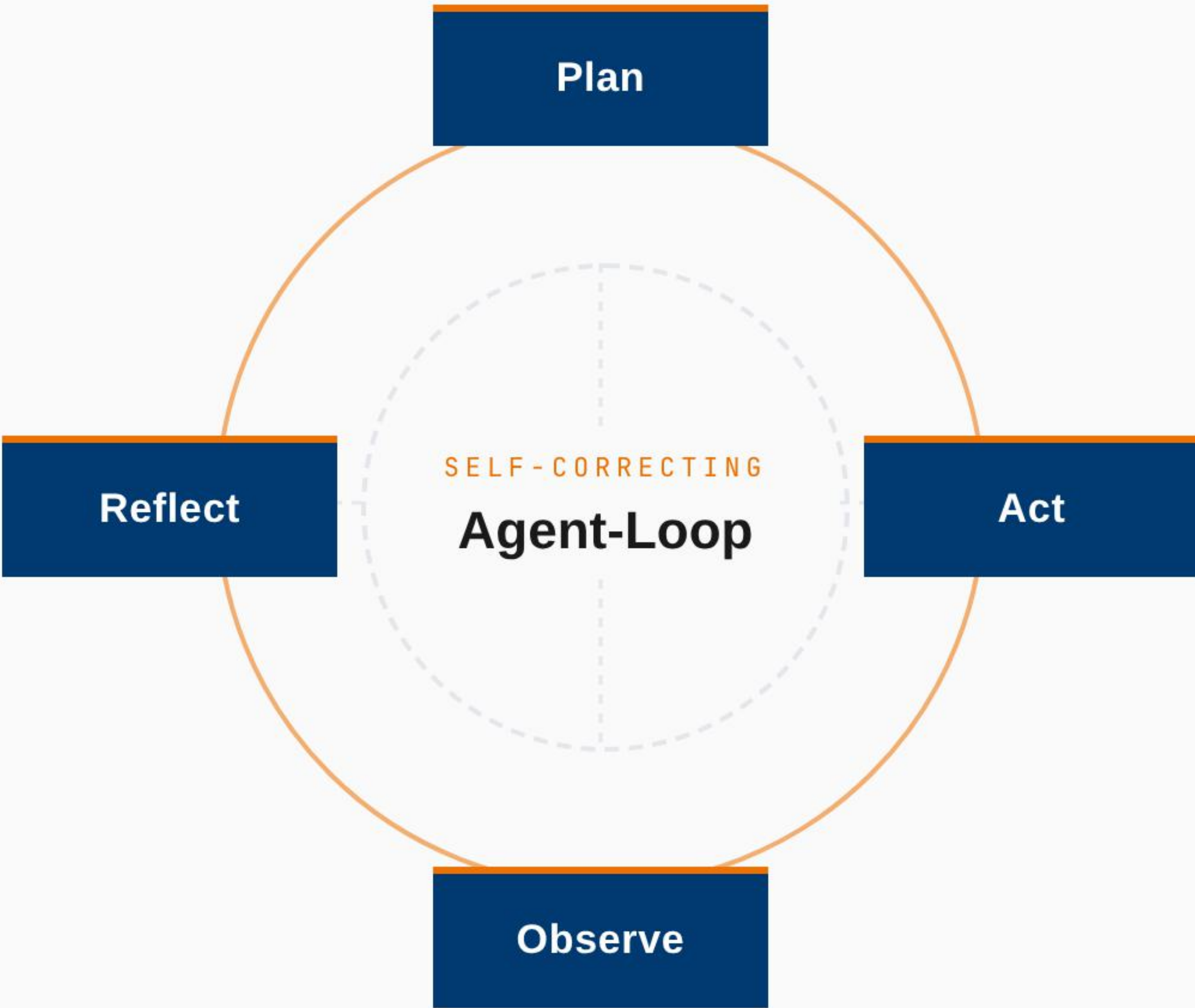
kein maschinell prüfbares Kriterium → keine Vollautonomie

VORAUSSETZUNG 2

Reversibilität

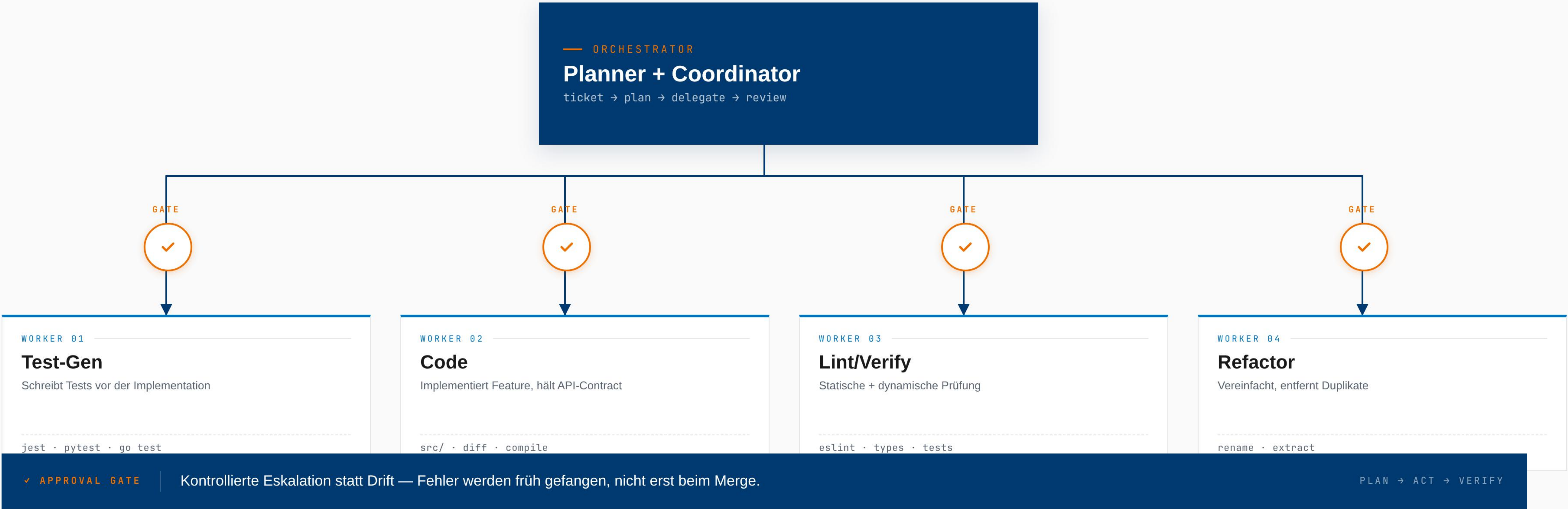
git revert, DB-Tx, Sandbox-Branches — irreversible Ops bleiben gegated

CLOSED-LOOP-SYSTEM



ORCHESTRATOR + SUBAGENTS MIT APPROVAL GATES.

Ein Hauptagent plant und delegiert. Vier spezialisierte Subagenten arbeiten parallel. Nach jedem Schritt steht ein Approval Gate.



PRAXIS · ECHTES LOG

VIER ISSUES, AUTONOM — IN EINER SITZUNG.

Hauptagent spawnt Fix- und Close-Subagenten je Issue. Mensch greift nur am Branch-Ende ein.

```
claude-code — orchestrator · ~/andritz/repo · zsh

Issues 15-18 confirmed open. Starting with issue 15 — delegating to a sonnet subagent.
● Agent(Fix issue #15) Sonnet 4.6
  └─ Done (56 tool uses · 98.8k tokens · 7m 11s)
Issue 15 implementation done. Now spawning close-issue subagent to commit.
● Agent(Close issue #15) Sonnet 4.6
  └─ Done (7 tool uses · 16.7k tokens · 1m 40s)
Issue 15 ✓ committed cleanly. Moving to issue 16.
● Agent(Fix issue #16) Sonnet 4.6
  └─ Done (44 tool uses · 106.1k tokens · 6m 39s)
Issue 16 implementation done. Spawning close-issue subagent.
● Agent(Close issue #16) Sonnet 4.6
  └─ Done (9 tool uses · 16.9k tokens · 34s)
Issues 15 and 16 done. Now starting issue 17.
● Agent(Fix issue #17) Sonnet 4.6
  └─ Done (65 tool uses · 75.4k tokens · 11m 30s)
● Agent(Close issue #17) Sonnet 4.6
  └─ Done (10 tool uses · 18.8k tokens · 41s)
Issue 17 ✓. Moving to the final issue 18.
● Agent(Fix issue #18) Sonnet 4.6
  └─ Done (67 tool uses · 95.0k tokens · 5m 57s)
```

SESSION-BILANZ

4 GitHub Issues · 8 Subagenten · 1 PR

Pro Issue: Fix-Subagent → Close-Subagent → Commit. Orchestrator hält den Plan, Subagenten halten den Kontext.

318 Tool-Aufrufe	449k Tokens insgesamt
36 min Wall-Clock	0 Manuelle Eingriffe

PATTERN: FIX → CLOSE → MOVE ON

Saubere Subagent-Grenzen halten den Kontext pro Issue klein — keine Drift zwischen Issues.

PRAXIS · ERKENNTNISSE

EIN PROMPT, VIER SCHRITTE, FERTIGER PR.

Pick the right scope — Autonomie-Level an Task-Komplexität koppeln.

ORCHESTRATOR-PROMPT

"You are an orchestrator — plan and make decisions only. Spawn subagents to do the coding, testing and validation. Resolve GitHub issues 15–18 and fix them fully agentially."

01

Orchestrator nimmt Prompt auf

02

Subagents implementieren + testen

03

Approval nach jeder Task

04

PR zur finalen Review

✓ Funktioniert

Klar abgegrenzter Scope mit definiertem Erfolgskriterium → läuft autonom durch.

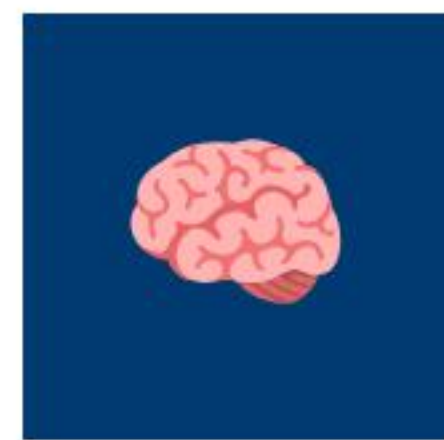
× Funktioniert nicht

Open-ended Architektur, neuartige Algorithmen → Agent driftet oder zirkelt.

ÜBERSICHT

VIER PATTERNS FÜR VIBE CODING.

Wiederverwendbares Erfahrungswissen — lehrbar und skalierbar.



PATTERN 01

Thinking Delegation

Auslagerung von Denkarbeit (Planung, Analyse) an einen spezialisierten Agenten. Hauptagent bleibt fokussiert.



PATTERN 02

Review & TDD

KI schreibt Tests zuerst, dann Implementation. Mensch reviewt strategisch an Checkpoints.



PATTERN 03

Junior Developer

KI wird wie ein Junior behandelt — klare Aufgaben, eng definierter Scope, regelmäßige Reviews.



PATTERN 04

Fix Point Algorithm

Iterative Verbesserung bis zur Stabilität — KI verfeinert, bis kein Fortschritt mehr erzielt wird.



VERTIEFUNG

EIN BEISPIEL JE PATTERN.

Pro Pattern: eine konkrete Anwendung und die beobachtete Wirkung.

PATTERN	BEISPIEL	WIRKUNG
Thinking Delegation	Komplexe Architekturentscheidung an Plan-Agenten delegiert	→ Hauptkontext bleibt schlank, bessere Entscheidungen
Review & TDD	Migrations-Tests vor Code generiert, dann Implementation	→ Regression vermieden, hohe Vertrauensbasis
Junior Developer	Routineaufgaben (z. B. Schema-Updates) klar abgegrenzt	→ Hohe Geschwindigkeit ohne Kontrollverlust
Fix Point	Iteratives Refactoring bis Linter/Tests stabil bleiben	→ Konvergenz statt unkontrollierter Drift

KERNBOTSCHAFT

Patterns machen Vibe Coding **lehrbar** und **skalierbar**.

PRIVAT-PROJEKTE · MINH CUONG TRAN

VIBE CODING AUSSERHALB DES BÜROS.

Zwei reale, im Einsatz befindliche Open-Source-Projekte aus der Schach-Community.



TEMPOMATE

Professionelle Schachuhr

Web-basiert, kein Download. Im Einsatz beim 150-jährigen Jubiläum der Schachfreunde Göppingen e.V.



URL
cuong.net/tempomate

TURNIER-DASHBOARD

Stadtmeisterschaft 2026	Runde 5 / 7	Live
Vereinsturnier Open	Runde 3 / 9	Live
U18 Landesmeisterschaft	Gepłant	15.06
Schnellschach-Cup	Gepłant	22.06

OPENPAIRING

Schachturnier-Verwaltung

Open-Source Turnier-Software für Schachvereine — vollständig durch Vibe Coding entwickelt.



URL
openpairing.org

DEMO · TEMPOMATE

DIE SCHACHUHR IM EINSATZ.

Web-basiert, kein Download — direkt im Browser öffnen und spielen.

Keine Installation

Direkt im Browser — kein Download, kein App Store.

Alle Zeitformate

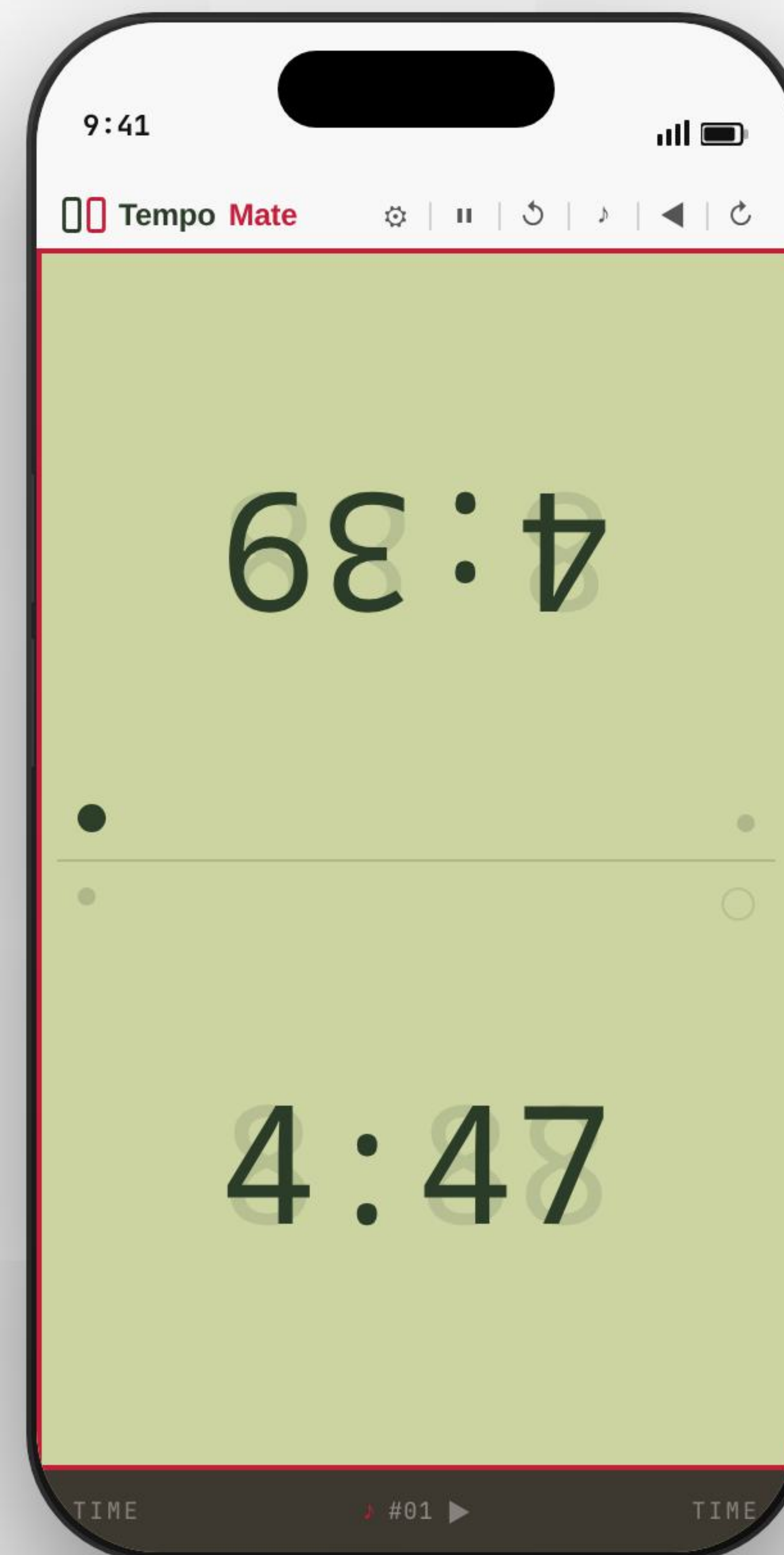
Klassisch, Fischer, Bronstein — vollständig konfigurierbar.

Turnier-erprobt

150-jähriges Jubiläum der Schachfreunde Göppingen.

Vibe Coding in Praxis

Von der Idee bis zum produktiven Einsatz — mit Claude entwickelt.



JETZT AUSPROBIEREN



URL

cuong.net/tempomate

ENTSTEHUNG

Von der Idee bis zum produktiven Einsatz — vollständig mit Vibe Coding entwickelt.

TECH STACK

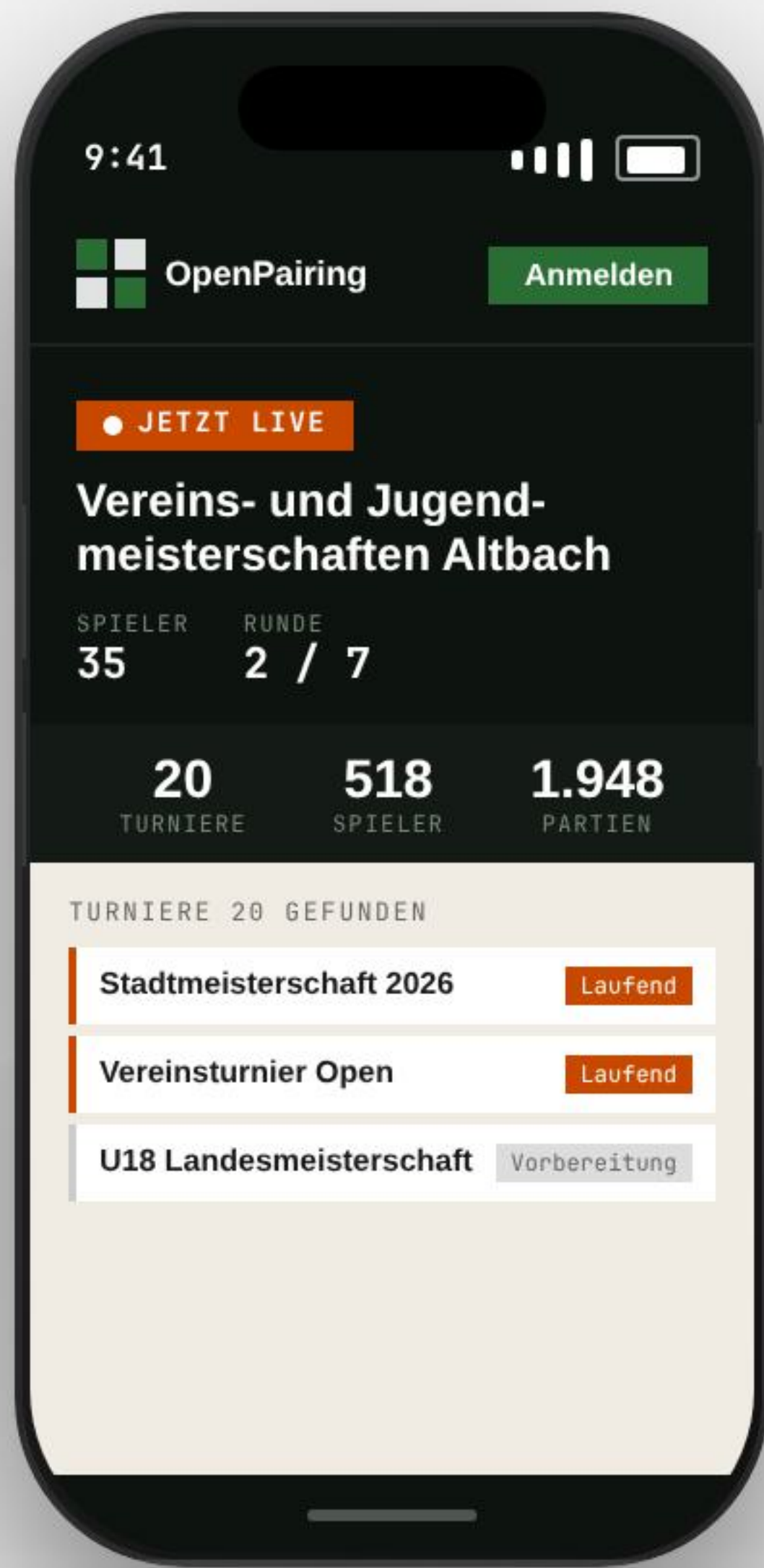
React · TypeScript · PWA

Kein Backend — läuft vollständig im Browser.

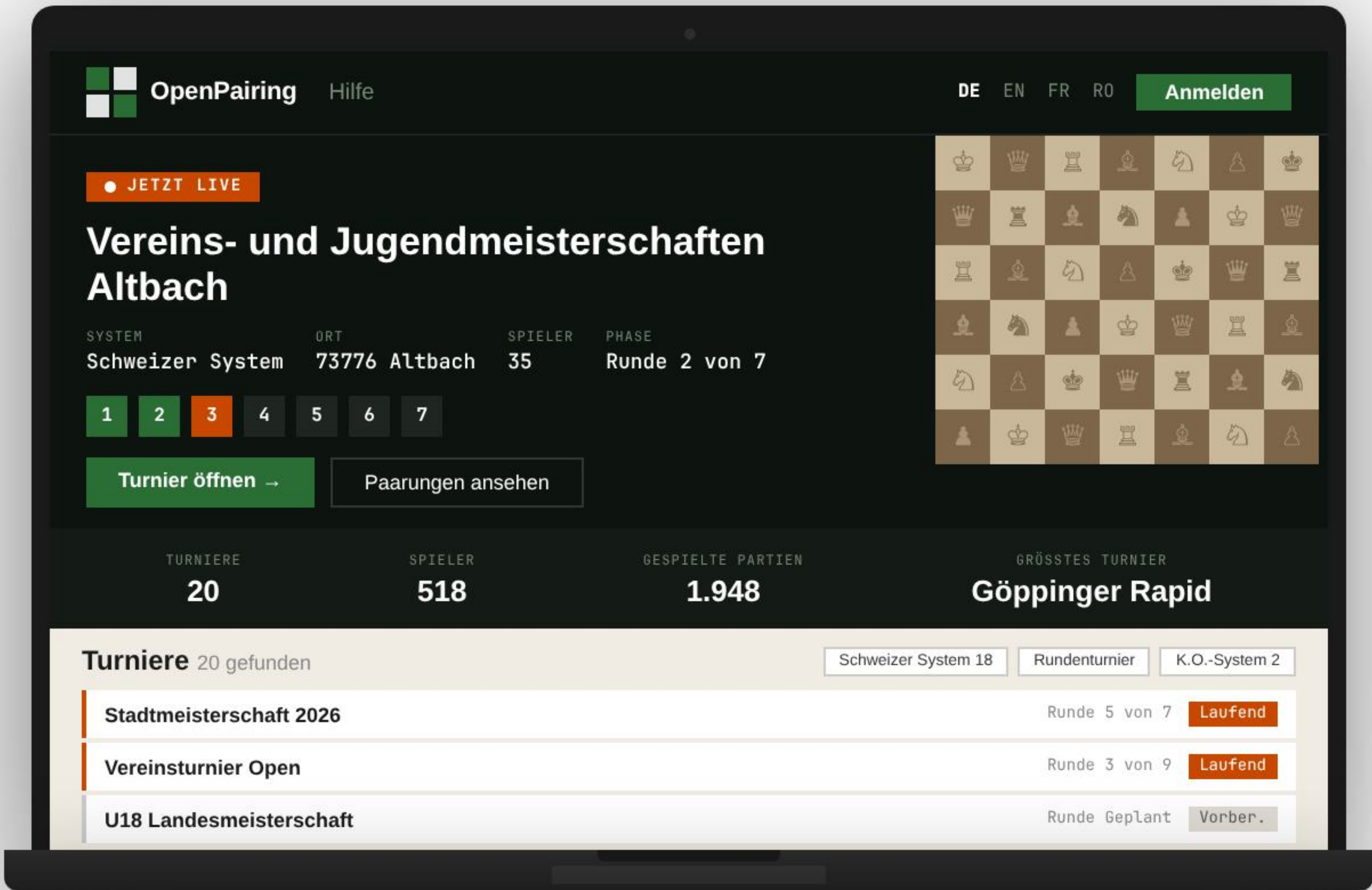
DEMO · OPENPAIRING

SCHACHTURNIER-VERWALTUNG. ÜBERALL.

Web-basiert, mehrsprachig, Open Source — auf jedem Gerät sofort einsatzbereit.



SMARTPHONE



DESKTOP / LAPTOP



TV / DISPLAY



URL
openpairing.org

FAZIT

FÜNF ERKENNTNISSE.

Was wir aus 10+ Enterprise-Projekten und drei tiefen Use Cases gelernt haben.

01 Vibe Coding ist eine Arbeitsweise — kein Werkzeug.

Wiederverwendbare Patterns machen sie lehrbar und skalierbar.

02 Vollagentische Workflows sind heute möglich.

Voraussetzung: Tool-Kompetenz, Engineering-Reife, klare Grenzen.

03 Domänenübergreifend einsetzbar.

Von Legacy-Modernisierung bis KI-Bildverarbeitung.

04 Mensch + KI, nicht „Mensch oder KI“.

Erfahrene Entwickler werden wichtiger — nicht überflüssig.

05 3× bis 20× Beschleunigung nachweisbar.

Wissenschaftlich untermauert · Bachelor-Arbeit, HS Esslingen.

—
FRAGEN?

VIELEN DANK.

Wir freuen uns auf den Austausch — Vibe Coding lebt vom Dialog.

—
VIBE CODING PATTERNS

Minh Cuong Tran

ANDRITZ Schuler

—
FULLY AGENTIC DEV

Jakob Ayo

Hochschule Esslingen

—
RISSERKENNUNG MIT KI

Viet Pham

Universität Tübingen